Control Structures

Without control structures, a program would just be a series of instructions executed in sequential order. This is fine for very simple programs, but most programs, like the driving directions example, aren't that simple. The driving directions included statements like, *Continue on Main Street until you see a church on your right* and *If the street is blocked because of construction*…. These statements are known as *control structures*, and they change the flow of the program's execution from a simple sequential order to a more complex and more useful flow.

If-Then-Else

In the case of our driving directions, Main Street could be under construction. If it is, a special set of instructions needs to address that situation. Otherwise, the original set of instructions should be followed. These types of special cases can be accounted for in a program with one of the most natural controlstructures: the *if-then-else structure*. In general, it looks something like this:

```
If (condition) then
{
  Set of instructions to execute if the condition is met;
}
Else
{
   Set of instruction to execute if the condition is not met;
}
```

For this book, a C-like pseudo-code will be used, so every instruction will end with a semicolon, and the sets of instructions will be grouped with curly braces and indentation. The if-then-else pseudo-code structure of the preceding driving directions might look something like this:

```
Drive down Main Street;
If (street is blocked)
{
```

```
    Turn right on 15th Street;
    Turn left on Pine Street;
    Turn right on 16th Street;
  }
  Else
  {
    Turn right on 16th Street;
  }
```

Each instruction is on its own line, and the various sets of conditional instructions are grouped between curly braces and indented for readability. In C and many other programming languages, the `then` keyword is implied and therefore left out, so it has also been omitted in the preceding pseudo-code.

Of course, other languages require the `then` keyword in their syntax— BASIC, Fortran, and even Pascal, for example. These types of syntactical differences in programming languages are only skin deep; the underlying structure is still the same. Once a programmer understands the concepts these languages are trying to convey, learning the various syntactical variations is fairly trivial. Since C will be used in the later sections, the pseudo code used in this book will follow a C-like syntax, but remember that pseudo-code can take on many forms.

Another common rule of C-like syntax is when a set of instructions bounded by curly braces consists of just one instruction, the curly braces are optional. For the sake of readability, it's still a good idea to indent these instructions, but it's not syntactically necessary. The driving directions from before can be rewritten following this rule to produce an equivalent piece of pseudo-code:

```
  Drive down Main Street;
  If (street is blocked)
  {
    Turn right on 15th Street;
    Turn left on Pine Street;
    Turn right on 16th Street;
  }
  Else
    Turn right on 16th Street;
```

This rule about sets of instructions holds true for all of the

control structures mentioned in this book, and the rule itself can be described in pseudo-code.

```
If (there is only one instruction in a set of instructions)
  The use of curly braces to group the instructions is optional;
Else
{
  The use of curly braces is necessary;
  Since there must be a logical way to group these instructions;
}
```

Even the description of a syntax itself can be thought of as a simple program. There are variations of if-then-else, such as select/case statements, but the logic is still basically the same: If this happens do these things, otherwise do these other things (which could consist of even more if-then statements).

While/Until Loops

Another elementary programming concept is the while control structure, which is a type of loop. A programmer will often want to execute a set of instructions more than once. A program can accomplish this task through looping, but it requires a set of conditions that tells it when to stop looping, lest it continue into infinity. A *while loop* says to execute the following set of instructions in a loop *while* a condition is true. A simple program for a hungry mouse could look something like this:

```
While (you are hungry)
{
  Find some food;
  Eat the food;
}
```

The set of two instructions following the while statement will be repeated *while* the mouse is still hungry. The amount of food the mouse finds each time could range from a tiny crumb to an entire loaf of bread. Similarly, the number of times the set of instructions in the while statement is executed changes depending on how much food the mouse finds.

Another variation on the while loop is an until loop, a syntax that is available in the programming language Perl (C doesn't use this syntax). An *until loop* is simply a while loop with the conditional statement inverted. The same mouse program using an until loop would be:

```
Until (you are not hungry)
{
  Find some food;
  Eat the food;
}
```

Logically, any until-like statement can be converted into a while loop. The driving directions from before contained the statement *Continue on Main Street until you see a church on your right*. This can easily be changed into a standard while loop by simply inverting the condition.

```
While (there is not a church on the right)
   Drive down Main Street;
```

For Loops

Another looping control structure is the *for loop*. This is generally used when a programmer wants to loop for a certain number of iterations. The driving direction *Drive straight down Destination Road for 5 miles* could be converted to a for loop that looks something like this:

```
For (5 iterations)
  Drive straight for 1 mile;
```

In reality, a for loop is just a while loop with a counter. The same statement can be written as such:

```
Set the counter to 0;
While (the counter is less than 5)
{
  Drive straight for 1 mile;
  Add 1 to the counter;
}
```

The C-like pseudo-code syntax of a for loop makes this even more apparent:

```
For (i=0; i<5; i++)
  Drive straight for 1 mile;
```

In this case, the counter is called `i`, and the for statement is broken up into three sections, separated by semicolons. The first section declares the counter and sets it to its initial value, in this case 0. The second section is like a while statement using the counter: *While* the counter meets this condition, keep looping. The third and final section describes what action should be taken on the counter during each iteration. In this case, `i++` is a shorthand way of saying, *Add 1 to the counter called i*.

Using all of the control structures, the driving directions from What Is Programming? can be converted into a C-like pseudo-code that looks something like this:

```
Begin going East on Main Street;
While (there is not a church on the right)
  Drive down Main Street;
If (street is blocked)
{
  Turn right on 15th Street;
  Turn left on Pine Street;
  Turn right on 16th Street;
}
Else
  Turn right on 16th Street;
Turn left on Destination Road;
For (i=0; i<5; i++)
  Drive straight for 1 mile;
Stop at 743 Destination Road;
```